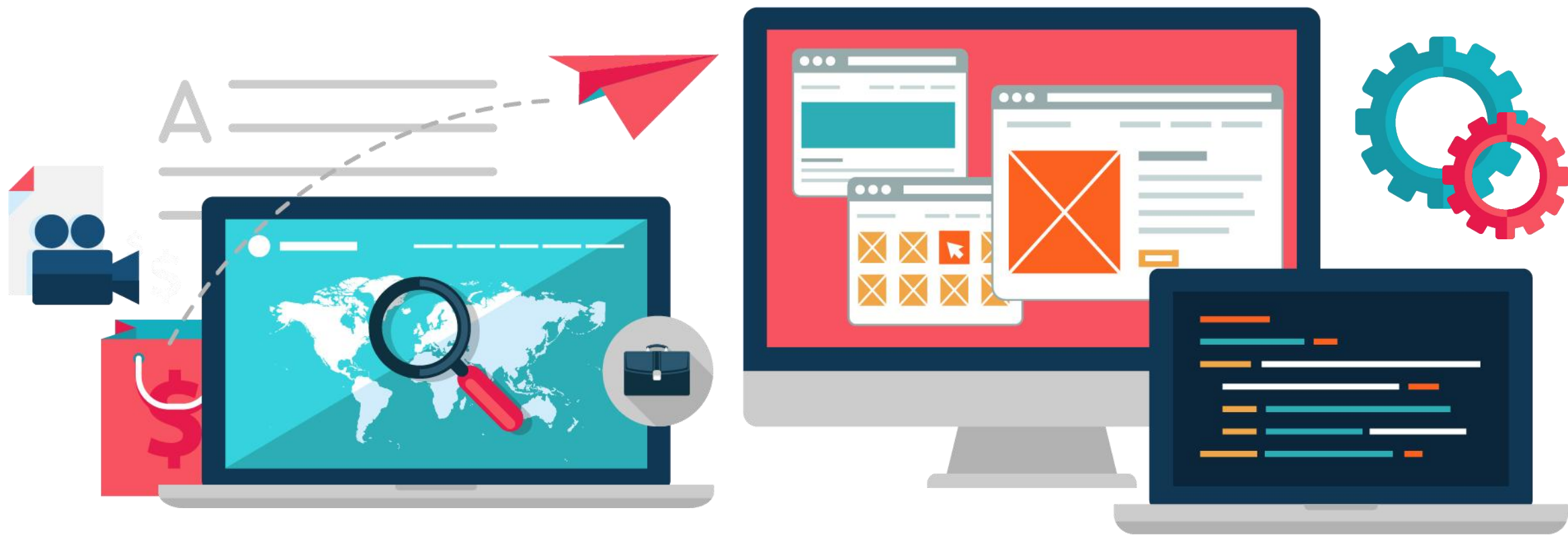


面向对象程序设计

Object Oriented Programming





多态性及其应用



01

静态多态

02

动态多态

03

虚机制的作用

04

虚机制的应用

05

虚机制的不足





多态性：相同的消息请求，执行不同的代码体，从而有不同的**行为过程**。

◆ 静态多态：根据目标对象的**静态类型**和参数表中参数的**静态类型**确定目标代码体。

- 函数重载
- 模板-不同的模板参数

◆ 动态多态



```
//静态多态-函数重载-例
#include <iostream>
using namespace std;
class A {
public:
    void f(int a) { cout << "Int"<<endl;}
    void f(char c) { cout << "Char"<<endl;}
};

int main( ){
    A obj;
    obj.f( 2 ); //输出 Int
    obj.f( 'c' ); //输出 Char
    return 0;
}
```



```
//静态多态-模板-例
#include <iostream>
#include <typeinfo>
using namespace std;
template <typename T>
class My {
public:
    void f(const T& t)
        { cout << typeid(t).name() << endl; }
};

int main( ) {
    My<int>  obj1;
    My<float> obj2;
    obj1.f(2);    //输出 i
    obj2.f(2);    //输出 f
    return 0;
}
```



◆静态多态

- 函数重载
- 模板

◆动态多态

- 根据目标对象的**动态类型**和参数表中参数的**静态类型**确定目标代码体。（虚机制）
- 根据目标对象的**动态类型**和参数表中参数的**动态类型**确定目标代码体。（C++不支持）



C++ 中的动态多态性-例

```
class B;
class A
{
public:
    virtual ~A( ) { }
    virtual void f(A *) { cout<<1<<endl;}
    virtual void f(B *) { cout<<2<<endl; }
};

class B:public A
{
public:
    virtual ~B( ){ }
    virtual void f(A *) { cout<<3<<endl;}
    virtual void f(B *) { cout<<4<<endl;}
};
```

```
int main()
{
    B b;
    A*pa = &b;
    pa->f(&b); //4
    pa->f(pa); //3
    b.f(pa);    //3
    return 0;
}
```



- ◆ 虚机制是实现动态多态的一种方法;
- ◆ 作用-在保持客户端访问接口不变的前提下, 可以变更类的实现

```
class A {  
    public:  
        virtual ~A() {}  
        virtual void Func( )  
        { /* 实现1 */ }  
};
```

```
class Client {  
    public:  
        doWork ( A* pA )  
        { pA-> Func(); }  
};
```

```
class B:public A {  
    public:  
        virtual ~B() {}  
        virtual void Func( )  
        { /* 实现2 */ }  
};
```

```
class C:public A {  
    public:  
        virtual virtual ~C() {}  
        void Func( )  
        { /* 实现3 */ }  
};
```



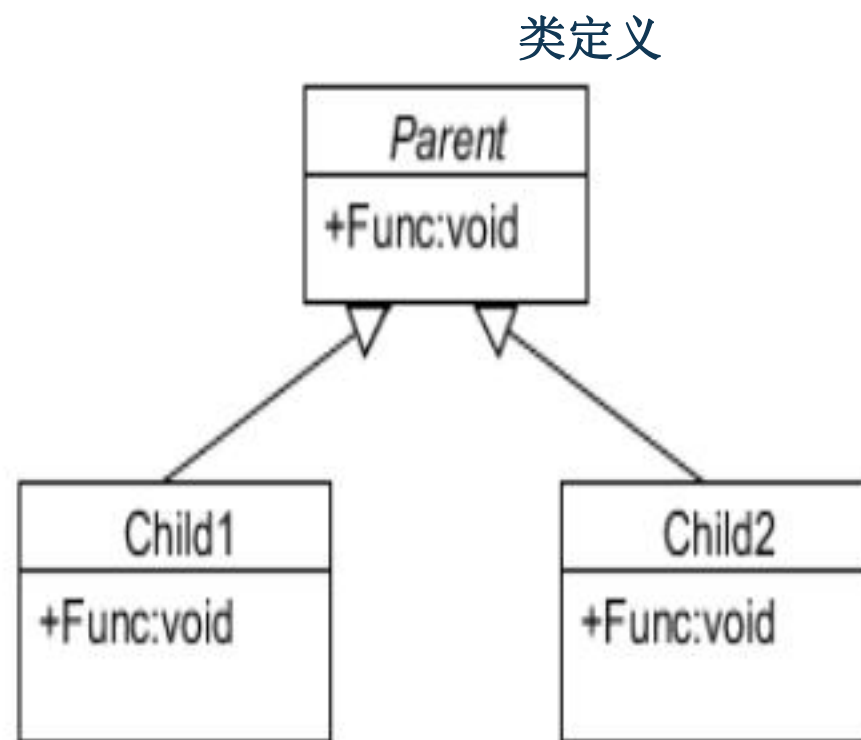
实质：通过子类型化来适应变化

```
void Proc(Parent * p)
{   p->Func();   }
```

```
Parent * p = new Child1;
Proc(p);
delete p;
```

```
Child2 myObj;
Parent& obj = myObj;
```

```
Proc(&obj);
```



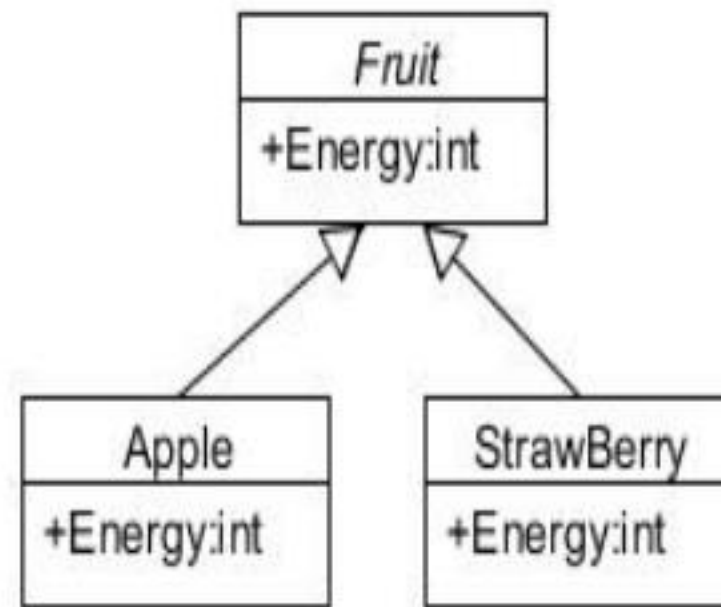


例1-父类提供接口，子类提供具体实现

客户端（使用端）

```
class Mouse { //部分
public:
    void Eat(Fruit & fruit)
        { weight += fruit.Energy( ) * 0.1; }
private:
    int weight;
}
```

类定义





例2--父类提供框架，子类提供细节

```
class AppSoftware
    :public Software
{ ... };
```

```
class WebSoftware
    : public Software
{ ... };
```

客户端（使用端）

```
void use(Software & sw )
{ sw.develop( ); sale(sw); }
```

```
Software* sw1 = new AppSoftware("众信");
Software* sw2 = new WebSoftware("千度");
```

```
class Software {
public:
    virtual ~Software( );
    void develop( ) {
        design( );
        coding( );
        test( );
        maintain();
    }
    virtual void design( )=0;
    virtual void coding( )=0;
    virtual void test( )=0;
    virtual void maintain( )=0;
};
```



例3--虚拟的拷贝构造

```
class A {  
public:  
    A(const A&);  
    ...  
};
```

```
class B:public A {  
public:  
    B(const B& b):A(b) {}  
    ...  
};
```

```
class C:public A {  
public:  
    C(const C& c):A(c) {}  
    ...  
};
```

```
A * copy( A & a ) {  
    //构造一个新的、与a一样  
    // 的同类型对象如何创建?  
    //常规做法  
    if ( a是B型对象)  
        return new B(*(B*)&a);  
    else if ( a是C型对象)  
        return new C(*(C*)&a);  
    else ....  
}
```



例3--虚拟的拷贝构造

问题：

1. 拷贝构造函数不能是虚的；
2. 即使能虚，各类中函数名字也不能相同

解决方案：新增加一个虚拟的同名函数，如**clone**

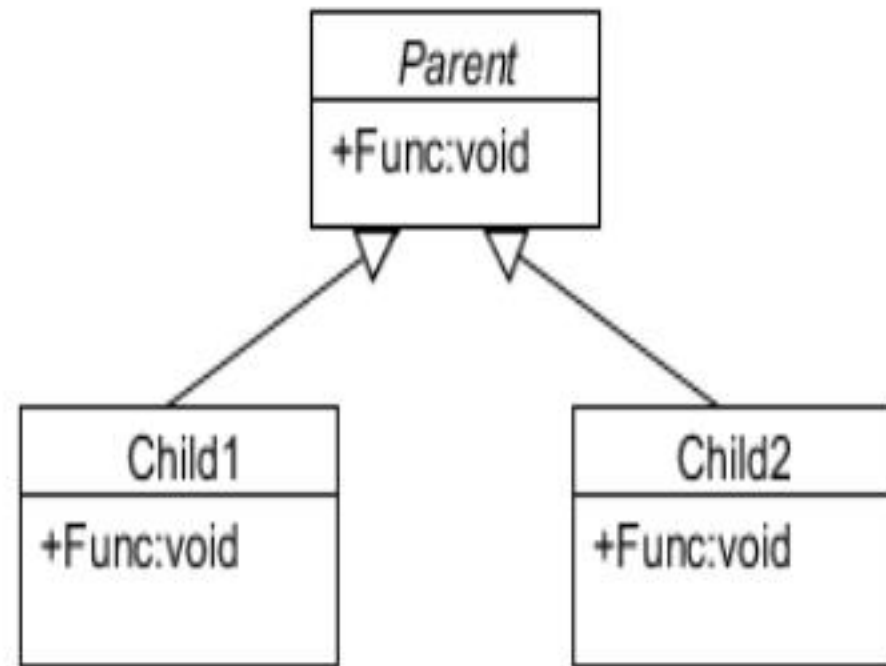
```
class A {  
public:  
    virtual ~A( );  
    virtual A * clone( )  
        { return new A(*this); }  
private: A(const A &) { }  
    ...  
};
```

```
class B:public A {  
public:  
    virtual B * clone( )  
        { return new B(*this); }  
private: B(const B &) { }  
    ...  
};
```

```
class C:public A {  
public:  
    virtual C * clone( )  
        { return new C(*this); }  
private: C(const C &){ }  
    ...  
};
```



```
class XXX {  
public:  
    void Do1(Parent & obj)  
        { obj.Func( ) ; }  
    void Do2( )  
        { pObj->Func( ) ; }  
    //初始化/设置pObj  
    XXX() :pObj(new Child1) {}  
    void SetParent(Parent * p)  
        { pObj = p;}  
private:  
    Parent * pObj;  
};
```



使用虚机制，使得当变更子类的具体实现时，不用改客户端**XXX**类的定义及实现



```
class Parent {  
public:  
    virtual ~Parent ();  
    virtual void F( );  
    virtual void G( );  
    virtual void H( );  
    ...  
};
```

举例： F()有两种不同的实现变化；
G()有三种不同的实现变化；
H()有四种不同的实现变化；

那么需要派生24个子类

所以，通常使用：
水平(关联、依赖)+继承

只使用虚机制，难以适应多个方向的变化。



使用关联、依赖+继承解决子类过多的问题(例)

```
class Parent {  
public:  
    Parent(ImpF *, ImpG *, ImpH * );  
    virtual ~Parent ();  
    void F( ) { pF->Func( ); }  
    void G( ){ pG->Func( ); }  
    void H( ){ pH->Func( ); }  
    ...  
private:  
    ImpF * pF;  
    ImpG * pG;  
    ImpH * pH;  
};
```

```
class ImpF {  
public:  
    virtual ~ImpF( ) {}  
    virtual void Func( )  
    { .... }  
};
```

← class ImpF1{...}

← class ImpF2{...}

```
class ImpG {  
public:  
    virtual ~ImpG( ) {}  
    virtual void Func( )  
    { .... }  
};
```

← class ImpG1{...}

← class ImpG2{...}

← class ImpG3{...}

```
class ImpH {  
public:  
    virtual ~ImpH( ) {}  
    virtual void Func( )  
    { .... }  
};
```

← class ImpH1{...}

← class ImpH2{...}

← class ImpH3{...}

← class ImpH4{...}



本章结束